

Final Project

Program Analysis and Verification, Spring 2024/25

Yarin Klempfner
ID: 212171706

Itamar Klartag
ID: 329601538

September 16, 2026

Throughout this project, $\mathbb{N}$ includes 0.

0 Static analysis structure

Each analysis presented in this project will follow this structure:

- Formal Definition
 - The concrete domain
 - Pointwise (variable) domain
 - Abstract state domain
- Lattice proof
- Galois connection
 - Define the Galois connection
 - Proof that the connection is Galois
- Abstract transfer functions
 - Definition
 - Proof of soundness
 - Proof of termination

1 Parity Analysis

Formal definition The concrete domain is made of states, composed of valuations $\sigma \in \Sigma = \text{Var} \rightarrow \mathbb{N}$. The concrete semantics of each command is:

$$\begin{aligned} \llbracket \text{skip} \rrbracket \sigma &= \{\sigma\} & \llbracket x := y + 1 \rrbracket \sigma &= \{\sigma[x \mapsto \sigma(y) + 1]\} \\ \llbracket x := y \rrbracket \sigma &= \{\sigma[x \mapsto \sigma(y)]\} & \llbracket x := y - 1 \rrbracket \sigma &= \{\sigma[x \mapsto \max(\sigma(y) - 1, 0)]\} \\ \llbracket x := K \rrbracket \sigma &= \{\sigma[x \mapsto K]\} & \llbracket \text{assume } E \rrbracket \sigma &= \{\sigma \mid \sigma \models E\} \\ \llbracket x := ? \rrbracket \sigma &= \{\sigma[x \mapsto n] \mid n \in \mathbb{N}\} \end{aligned}$$

and **assert** / join / meet are standard. The concrete domain is the collecting semantics over sets of such valuations:

$$C = (2^\Sigma, \subseteq, \cap, \cup),$$

a complete lattice (arbitrary unions/intersections of sets of valuations exist), with $\perp_C = \emptyset$ and $\top_C = \Sigma$.

Abstract domain overview We propose a disjunctive domain made of states, $A = \mathcal{P}(\text{Var} \rightarrow B)$. That is, the domain is the power set of all the mappings between the variables declared in the program and B , a set of abstract values $\{O, E, Z\}$. So, in a program with one variable, x , there are 8

possible states: $\{\emptyset, \{x \rightarrow O\}, \{x \rightarrow Z\}, \{x \rightarrow E\}, \{x \rightarrow E, x \rightarrow O\}, \{x \rightarrow E, x \rightarrow Z\}, \{x \rightarrow O, x \rightarrow Z\}, \{x \rightarrow E, x \rightarrow O, x \rightarrow Z\}\}$. These states follow the regular order ($\sqsubseteq$) of sets, $\subseteq$. Meaning that in our one-variable example, $\{x \rightarrow E, x \rightarrow O, x \rightarrow Z\}$ is the top ($\top$), and the empty set $\emptyset$ is the bottom ($\perp$). Hopefully, this overview of our analysis makes it easier to understand the following definitions and proofs.

Base set - single variable encoding (B) We define the lattice for a single variable. The set B is defined as: $B = \{Z, E, O\}$. The elements of B represent the mathematical sets $\{0\}$, $Even := \{2, 4, 6, \dots\}$ and $Odd := \{1, 3, 5, \dots\}$ (We changed the name to easily differentiate between the mathematical group and its abstract symbol). The reason we include the abstract representation of 0 separately (and not as a part of E) is due to the behavior for $i := j - 1$ when $j = 0$.

Abstract state domain - $\mathcal{P}(Var \rightarrow B)$ As stated above, the states in the lattice are a power set of the mappings between variables and abstract representations ($t := Var \rightarrow B$). The abstract domain, $A := \mathcal{P}(Var \rightarrow B)$ is ordered by $\subseteq$, with $\sqcup = \cup$, $\sqcap = \cap$, $\perp = \emptyset$ and $\top = B^{Var}$. Since the order, meet, and join are well-defined, and since the lattice has a finite amount of states, $2^{|Vars|}$, it is indeed a complete lattice, with a height of $3^{|Vars|}$.

Galois connection Now, we need to establish the Galois connections between the abstract states and the concrete states. As we already have C and A (A is the abstract domain that we defined above), we only have to define γ and α .

In order to define α , we will use a helper function, β , that works on a single integer. Formally:

$$\beta : \mathbb{N} \rightarrow B, \beta(n \in \mathbb{N}) = \begin{cases} E & n \in Even \\ O & n \in Odd \\ Z & n = 0 \end{cases}$$

and lift it to valuations: $\beta(\sigma) := \lambda x. \beta(\sigma(x))$, i.e. $\beta \circ \sigma : Var \rightarrow B$. With the helper function established, we can define the Galois connection:

$$\alpha(S) = \{\beta \circ \sigma \mid \sigma \in S\} \text{ and } \gamma(T) = \{\sigma \mid \beta \circ \sigma \in T\} = \bigcup_{t \in T} \gamma(t) \text{ where } \gamma(t) = \{\sigma \mid \forall x. \sigma(x) \in t(x)\}.$$

Meaning, α takes the mapping of variables to numbers and uses β to translate each mapping into a mapping from variable to abstract representation. As the abstract domain is disjunctive, over $\mathcal{P}(Var \rightarrow B)$, a state in the abstract domain can include the same variable mapped to different abstract representations.

In order to make the definitions clear, assume that we have a program that consists solely of $x = 5$. The concrete state would be $d = \{x \rightarrow 5\}$, and then $A = \alpha(d) = \{\{x \rightarrow O\}\}$, and $\gamma(\alpha(d)) = \gamma(A) = \{x \rightarrow 1, x \rightarrow 3, x \rightarrow 5, \dots\}$. As a sanity check, we can clearly see that $d \subseteq \gamma(\alpha(d))$, as required of a Galois connection. Proving it formally:

Prove: $\forall S \in C. \forall T \in A. \alpha(\gamma(T)) \subseteq T$ and $S \subseteq \gamma(\alpha(S))$.

Proof. Let $S \in C$ and $T \in A$. $\alpha(S) \subseteq T \iff \forall \sigma \in S. \beta \circ \sigma \in T \iff S \subseteq \gamma(T)$. Since this follows directly from the definitions, we arrive at $\alpha(S) \subseteq T \iff S \subseteq \gamma(T)$, thus proving that (α, γ) is indeed a Galois connection.

Abstract transfer function

After establishing the Galois connection, we can define the abstract transfer functions. The concrete transfer functions are defined in the project definition and are obvious.

ATF for the commands: The ATFs, for every subset t of $T \in A$:

$$\begin{aligned}
\llbracket \text{skip} \rrbracket^\#(t) &= \{t\} \\
\llbracket x := y \rrbracket^\#(t) &= \{t[x \mapsto t(y)]\} \\
\llbracket x := K \rrbracket^\#(t) &= \{t[x \mapsto \beta(K)]\} \\
\llbracket x := ? \rrbracket^\#(t) &= \{t[x \mapsto b] \mid b \in B\} \\
\llbracket x := y + 1 \rrbracket^\#(t) &= \{t[x \mapsto b] \mid b \in \text{succ}(t(y))\} \\
\llbracket x := y - 1 \rrbracket^\#(t) &= \{t[x \mapsto b] \mid b \in \text{pred}(t(y))\} \\
\llbracket \text{assume } E \rrbracket^\#(t) &= \{t\} \text{ if } t \models^? E, \text{ else } \emptyset \\
\llbracket \text{assert } ORC \rrbracket^\#(t) &= \{t\}, \text{Err if } t \not\models ORC
\end{aligned}$$

where $t \models^? E$ means that t may satisfy E . For example, if $E := x = y$, and $t := \{x \rightarrow E, y \rightarrow E\}$, then t may satisfy E , but if $t := \{x \rightarrow E, y \rightarrow O\}$ then it cannot.

The *succ* function is defined as:

$$\text{succ}(x) := \begin{cases} E & \text{if } x = O \\ O & \text{if } x = E \text{ or } x = Z \end{cases} \quad \text{And } \text{pred} \text{ is defined as: } \text{pred}(x) := \begin{cases} O & \text{if } x = E \\ Z & \text{if } x = Z \\ Z, E & \text{if } x = O \end{cases}$$

The *assert* keyword is used to verify the program, and thus has no effect on the domain; rather, it checks some mappings in the domain and throws an error when the conditions aren't satisfied.

A note for *assume* : the *assume* keyword does not change the state for *assume* $i = j$, *assume* $i \neq j$ unless i or j are mapped specifically to Z .

Soundness and termination: With the definition of the ATFs, the analysis definition is complete.

Now, we need to prove its soundness and the fact that it terminates.

To prove soundness, we must prove the soundness of the abstract transformers. A transformer is sound if it is monotone and meets the condition: $\forall d \in D. f(\gamma(d)) \subseteq \gamma(f^\#(d))$. We start with monotonicity. A function is monotone if:

$$\forall d_1, d_2 \in D. d_1 \sqsubseteq_D d_2 \Rightarrow f^\#(d_1) \sqsubseteq_D f^\#(d_2)$$

This property is self-evident for the vast majority of transformers. For example, all of the functions assigning a new value to a variable, like $i := j$, achieve the same result no matter what the original value of i was, so they are all monotone by definition. The only function remaining is *skip*, and since it does not change the domain, it is monotone.

Now that we proved that all ATFs are monotone, we must show that for all ATFs: $\forall T \in A. f(\gamma(T)) \subseteq \gamma(f^\#(T))$, for the appropriate concrete transformer f for each one. Since both the concrete and abstract transformers distribute over unions, it suffices to prove the property on $T = \{t\}$.

For *skip*, the property is evident. For $i := ?$, both the abstract and concrete transformers result in i

being mapped to $\top$, and since $\gamma(\top) = \top \subseteq \top$, the property holds.

For $i := j$ and $i := K$, they behave the same when $j \rightarrow p_j \in \{E, O, Z\}$ ($\beta(K) \in \{Z, O, E\}$ always). W.L.O.G., we assume that $p_j = O$ (and $K = O$). $f(\gamma(t)) = f(t[j \rightarrow 1, j \rightarrow 3, \dots]) = t[i \rightarrow 1, j \rightarrow 1, i \rightarrow 3, j \rightarrow 3, \dots]$. And $\gamma(f^\#(t)) = \gamma(t[i \rightarrow O, j \rightarrow O]) = t[i \rightarrow 1, j \rightarrow 1, i \rightarrow 3, j \rightarrow 3, \dots]$. Thus, $f(\gamma(t)) = \gamma(f^\#(t))$, which includes $f(\gamma(t)) \subseteq \gamma(f^\#(t))$, as required. Since $\top$ and $\perp$ transform into $\top$ and $\perp$ accordingly using the Galois connection, we can conclude that both $i := j$ and $i := K$ are sound.

As for $i := j + 1$, if $d[j \rightarrow \perp]$ or $d[j \rightarrow \top]$, the function is identical to $i := j$, and thus sound. W.L.O.G., assume $d[j \rightarrow O]$. Then: $f(\gamma(d)) = f(d[i \rightarrow p_i, j \rightarrow 1, j \rightarrow 3, \dots]) = d[i \rightarrow 2, j \rightarrow 1, i \rightarrow 4, j \rightarrow 3, \dots]$. And $\gamma(f^\#(d)) = \gamma(d[i \rightarrow E, j \rightarrow O]) = d[i \rightarrow 2, j \rightarrow 1, i \rightarrow 4, j \rightarrow 3, \dots]$. Thus, $f(\gamma(d)) = \gamma(f^\#(d))$, which satisfies the requirement, so $i := j + 1$ is sound.

The transform $i := j - 1$ behaves identically when $d[j \rightarrow \top]$ or $d[j \rightarrow \perp]$. However, when $d[j \rightarrow E]$, we get: $f(\gamma(d)) = f(d[j \rightarrow 2, j \rightarrow 4, \dots]) = d[i \rightarrow 1, j \rightarrow 2, i \rightarrow 3, j \rightarrow 4, \dots]$. And $\gamma(f^\#(d)) = \gamma(d[i \rightarrow O]) = d[i \rightarrow 1, j \rightarrow 2, i \rightarrow 3, j \rightarrow 4, \dots]$. Evidently, $f(\gamma(d)) \subseteq \gamma(f^\#(d))$ is satisfied. When $d[j \rightarrow Z]$, $f(\gamma(d)) = f(d[j \rightarrow 0]) = d[i \rightarrow 0, j \rightarrow 0]$, matching $\gamma(f^\#(d)) = \gamma(d[i \rightarrow Z])$.

The Odd case, $d[j \rightarrow O]$ is a bit different. Assuming that, we get $f(\gamma(d)) = f(d[i \rightarrow 1, i \rightarrow 3, \dots]) = d[i \rightarrow 0, i \rightarrow 2, \dots]$. However, for $\gamma(f^\#(d))$ the case is a bit different since $f^\#$ uses *pred*. We get: $\gamma(f^\#(d)) = \gamma(\{d[i \rightarrow E, j \rightarrow O], d[i \rightarrow Z, j \rightarrow O]\}) = \gamma(d[i \rightarrow E, j \rightarrow O]) \cup \gamma(d[i \rightarrow Z, j \rightarrow O]) = d[i \rightarrow 2, j \rightarrow 1, i \rightarrow 4, j \rightarrow 3, \dots] \cup d[i \rightarrow 0, j \rightarrow 1, j \rightarrow 3, \dots]$. Thus, we conclude that $f(\gamma(d)) \subseteq \gamma(f^\#(d))$, as required. Thus, $i := j - 1$ is sound.

Since all the expressions mimic the commands, they are also sound. Note that the expressions $i \neq K$ and $i \neq j$ don't do anything in the abstract domain if they cannot definitively eliminate a class. Thus, all the expressions are sound, and consequently, the branching from the *assume* keyword is sound.

Since all of the functions are sound on singular variables, we can expand the property to pointwise on the entire domain, meaning that we proved that all the transfer functions are sound, and thus the analysis is sound.

Lastly, we need to prove termination. As we already demonstrated, the functions are monotone, so we need to prove that they satisfy the Ascending Chain Condition (ACC). For one variable, the lattice that we have chosen has a finite height. A state can ascend a finite amount of times before reaching $\top$. That puts a limit on the length of an ascending chain, making it finite. Since the functions are monotone, any ascending chain created using the functions is finite, confirming the ACC and resulting in termination for all functions.

Running example consider the example code provided in the project description:

Source	Command	Target
L_0	$n := ?$	L_1
L_1	$i := 0$	L_2
L_2	$j := 0$	L_3
L_3	assume ($i = n$)	L_6
L_3	assume ($i \neq n$)	L_4
L_4	$i := j + 1$	L_5
L_5	$j := i$	L_3
L_6	assert (ODD i ODD j) (EVEN i EVEN j)	L_7
L_7	assert (ODD i ODD n) (EVEN j EVEN n)	L_8

(Throughout this example, the abstract vectors are written using the format (n, i, j) , e.g. (E, Z, O) means $n = E, i = Z, j = O$)

The entry state is $\text{state}[L_0] = \top_A = B^{\{n, i, j\}}$, all 27 vectors.

The first command at L_0 , $\llbracket n := ? \rrbracket^\#$, applied to $\top_A$, still yields all 27 vectors, so $\text{state}[L_1] = \top_A$.

$\llbracket i := 0 \rrbracket^\#$ sets $i \mapsto \beta(0) = Z$ in every vector of $\text{state}[L_1]$, collapsing the 3 choices for i down to Z and leaving n, j free:

$$\text{state}[L_2] = \{Z, E, O\} \times \{Z\} \times \{Z, E, O\}, \quad |\text{state}[L_2]| = 9.$$

(n, j can still be any option, but i is collapsed into Z)

Similarly, $\llbracket j := 0 \rrbracket^\#$ collapses $j \mapsto Z$, giving the state that first flows into the loop head L_3 :

$$S_0 := \{(Z, Z, Z), (E, Z, Z), (O, Z, Z)\} = \{Z, E, O\} \times \{Z\} \times \{Z\}.$$

Then, we go to L_3 , where the loop starts. In the following table, we will show what vector is added after every iteration of the loop (The "loop" doesn't actually run, we re-run the analysis with the existing states until we reach a fixpoint)

So, the fixpoint is:

Round	New at L_3 (by $\sqcup$)	How it was produced
0	$(Z, Z, Z), (E, Z, Z), (O, Z, Z)$	flows in directly from L_2
1	$(E, O, O), (O, O, O)$	assume ($i \neq n$) keeps $(E, Z, Z), (O, Z, Z) (\rightarrow L_4)$; $i := j + 1$: $\text{succ}(Z) = O \Rightarrow (E, O, Z), (O, O, Z) (\rightarrow L_5)$; $j := i$: $(E, O, Z) \mapsto (E, O, O), (O, O, Z) \mapsto (O, O, O)$
2	$(E, E, E), (O, E, E)$	assume ($i \neq n$) keeps $(E, O, O), (O, O, O) (\rightarrow L_4)$; $i := j + 1$: $\text{succ}(O) = E \Rightarrow (E, E, O), (O, E, O) (\rightarrow L_5)$; $j := i$: $(E, E, O) \mapsto (E, E, E), (O, E, O) \mapsto (O, E, E)$
3	<i>none</i> (fixpoint)	assume ($i \neq n$) on $(E, E, E), (O, E, E)$ keeps both $(\rightarrow L_4)$; $i := j + 1$ gives $\text{succ}(E) = O \Rightarrow (E, O, E), (O, O, E)$; $j := i$ gives $(E, O, O), (O, O, O)$ – already in $\text{state}[L_3]$.

$$\text{state}[L_3] = \{(Z, Z, Z), (E, Z, Z), (O, Z, Z), (E, O, O), (O, O, O), (E, E, E), (O, E, E)\},$$

, i.e 7 vectors. Now, after reaching the loop's fixpoint, we have a branch, of **assume** $i = n$ and **assume** $i \neq n$. Applying $\llbracket \text{assume}(i = n) \rrbracket^\#$ to $\text{state}[L_3]$ keeps exactly the vectors where i 's class equals

n 's class:

$$\text{state}[L_6] = \{(Z, Z, Z), (E, E, E), (O, O, O)\},$$

and it is clear that all of them satisfy the assertions. Applying $\llbracket \text{assume}(i \neq n) \rrbracket^\#$ keeps every vector except (Z, Z, Z) (the only one where $i \neq n$ is abstractly impossible, since both are forced to be exactly 0). While consisting of more vectors, it is still plain to see that all of them satisfy the assertions, thus our analysis verifies the program, as required.

2 Shape Analysis of Unshared Acyclic Lists

2.1 Formal definition

The goal of this analysis is to verify memory safety, acyclicity, and the absence of sharing in linked lists by abstracting unbounded heaps into finite shape graphs.

The concrete domain The concrete domain is made of heaps, which are triples $h = (L, \text{env}, n)$ where $L \subseteq \text{Loc}$ is a finite set of allocated memory cells, $\text{env} : \text{Var} \rightarrow L \cup \{\text{null}\}$ maps program variables to these cells, and $n : L \rightarrow L \cup \{\text{null}\}$ represents the successor pointer field. A heap is well formed if it is unshared (meaning $n(c) = n(c') \neq \text{null} \implies c = c'$) and acyclic. The initial state is the empty heap with all variables set to null.

Abstract domain overview We propose an abstraction where the numbers of anonymous, unaliased cells in a heap segment are abstracted by the lattice $\mathcal{L} = \{1, O, E\}$, which represents exactly 1 step, an odd number of steps $\{3, 5, 7, \dots\}$, and an even number of steps $\{2, 4, 6, \dots\}$ respectively. A shape graph is defined as $G = (N, \text{env}, \text{succ}, \text{len}, \text{gp})$ where N is a finite set of named nodes, env is surjective on N (meaning every node is pointed to by at least one variable), succ is the abstract successor function between named nodes, $\text{len} : N \rightarrow \mathcal{L}$ abstracts the segment lengths between them, and $\text{gp} : N \rightarrow \{tt, ff\}$ flags if a named node is pointed to by unreachable garbage (which is vital for detecting hidden sharing violations).

Abstract state domain - $\mathcal{P}(\mathcal{S})$ The abstract state domain A is $\mathcal{P}(\mathcal{S})$, where $\mathcal{S}$ is the finite set of all valid shape graphs up to isomorphism. It is ordered by $\subseteq$, with $\sqcup = \cup$, $\perp = \emptyset$, and $\gamma(T) = \bigcup_{G \in T} \gamma(G)$.

Galois connection We establish the Galois connection between the abstract states (shape graphs) and concrete heaps. We define $\alpha(H) = \{\beta(h) \mid h \in H\}$, where $\beta(h)$ is the unique shape graph constructed by taking the named cells as nodes, assigning the first named cell or null reached from each as its successor, defining the abstract length class as len , and recording unreachable predecessors as gp .

Prove: (α, γ) is a Galois insertion.

Proof. For every $G \in \mathcal{S}$, $\gamma(G) \neq \emptyset$ because concrete segment lengths can always be chosen from the abstract classes, and arbitrary garbage cells can be added for gp -flagged nodes. By definition, $\alpha(H) \subseteq T \iff \forall h \in H. \beta(h) \in T \iff H \subseteq \gamma(T)$. Furthermore, α preserves unions natively due to the disjunctive structure. Thus, (α, γ) successfully forms a Galois insertion.

Abstract transfer function The transformers on a shape graph G first conceptually modify the graph based on the command and then normalise it.

Normalisation ($\text{norm}(G)$) Because abstract shape graphs only represent named nodes, whenever a variable is reassigned, a node may lose its last pointing variable. The $\text{norm}(G)$ function

is a reduction algorithm that repeatedly removes any node u to which no variable points. It operates in two cases:

- **Merge:** If u has a predecessor p (i.e., $\text{succ}(p) = u$), we bypass u by setting $\text{succ}(p) := \text{succ}(u)$. We concatenate their segments by mathematically adding their lengths in the abstract lattice: $\text{len}(p) := \text{len}(p) \oplus \text{len}(u)$.
- **Garbage Collection:** If u has no predecessor, it is completely unreachable (a memory leak). We delete u , but if its segment was pointing to another node v ($v = \text{succ}(u) \neq \text{null}$), we must set $\text{gp}(v) := tt$ to correctly flag that the leaked segment still secretly points to v .

For every command, the ATFs are defined as follows (applying union over elements in sets):

$$\begin{aligned}
\llbracket x := \text{null} \rrbracket^\#(G) &= \text{norm}(G[\text{env}(x) \mapsto \text{null}]) \\
\llbracket x := y \rrbracket^\#(G) &= \text{norm}(G[\text{env}(x) \mapsto \text{env}(y)]) \\
\llbracket x := \text{new} \rrbracket^\#(G) &= \text{Add fresh node } w, \text{succ}(w) = \text{null}, \text{len}(w) = 1, \text{gp}(w) = ff, \text{ set } x \text{ to } w \\
\llbracket x := y.n \rrbracket^\#(G) &= \text{err if } \text{env}(y) = \text{null}; \text{ if } \text{len} = 1 \text{ set } x \text{ to } \text{succ}(\text{env}(y)) \\
\llbracket x.n := y \rrbracket^\#(G) &= \text{err if } \text{env}(x) = \text{null}; \text{ detach } \text{env}(x) \text{ and conditionally attach to } \text{env}(y) \\
\llbracket \text{assume } E \rrbracket^\#(G) &= \{G\} \text{ if } E \text{ evaluates to } tt \text{ in the split graph} \\
\llbracket \text{assert } ORC \rrbracket^\#(G) &= \{G\}, \text{Err if } ORC \text{ evaluates to } ff \text{ or dereferences null}
\end{aligned}$$

For $x := y.n$, if $\text{len}(\text{env}(y)) \in \{O, E\}$, the intermediate cell is materialised into a fresh node matching a length in $\text{len}(\text{env}(y)) \ominus 1$, and x is set to point to it. For $x.n := y$, an error is thrown if $\text{env}(x) = \text{null}$. To attach, if $y \neq \text{null}$, sharing is reported if $\text{env}(y)$ already has a predecessor or $\text{gp} = tt$, and a cycle is reported if $\text{env}(y) \rightarrow^* \text{env}(x)$.

The abstract addition $\oplus$ between length classes is commutative and defined as:

$$\begin{array}{lll}
1 \oplus 1 = E & 1 \oplus E = O & 1 \oplus O = E \\
E \oplus E = E & E \oplus O = O & O \oplus O = E
\end{array}$$

And the abstract subtraction $\ominus 1$ (which returns a set of possible classes) is defined as:

$$\begin{aligned}
O \ominus 1 &= \{E\} \\
E \ominus 1 &= \{1, O\}
\end{aligned}$$

Soundness and termination: With the definition of the ATFs, the analysis definition is complete. Now, we need to prove its soundness and the fact that it terminates.

To prove soundness, we must prove the soundness of the abstract transformers. A transformer is sound if it is monotone and meets the condition: $\forall d \in D. f(\gamma(d)) \subseteq \gamma(f^\#(d))$. We start with monotonicity. A function is monotone if: $\forall d_1, d_2 \in D. d_1 \sqsubseteq_D d_2 \Rightarrow f^\#(d_1) \sqsubseteq_D f^\#(d_2)$. This property is self-evident because our transformers are defined on individual shape graphs and lifted by union, meaning they are all monotone by definition.

Now that we proved that all ATFs are monotone, we must show that for all ATFs: $\forall T \in A. f(\gamma(T)) \subseteq \gamma(f^\#(T))$, for the appropriate concrete transformer f for each one. Since both the concrete and

abstract transformers distribute over unions, it suffices to prove the property on a single pre-normalised graph G' .

First, we establish a formal property for our normalisation function: $\gamma(G') \subseteq \gamma(\text{norm}(G'))$ for any pre-normalised graph G' . One step of normalisation removes an unnamed node u . If u has a predecessor p ($\text{succ}(p) = u$), the concrete path lengths concatenate: $m(p) \rightarrow^{k_1} m(u) \rightarrow^{k_2} m(\text{succ}(u))$. Because the abstract addition $\oplus$ is the exact image of concrete addition on the classes, $k_1 + k_2 \in \llbracket \text{len}(p) \oplus \text{len}(u) \rrbracket$, so reachability is exactly preserved and the heap mapping is unaffected. If u is unreachable, no reachable segment points to it, meaning its entire segment is structurally unreachable. The last cell of its segment still points to $m(\text{succ}(u))$, matching the abstract rule to set $gp(\text{succ}(u)) := tt$ after removal. Thus, normalisation is sound.

Now, we proceed by checking the commands using a bijection m mapping abstract named nodes to concrete cells. Let $h' = \llbracket C \rrbracket h$. By the normalisation lemma, it suffices to show $h' \in \gamma(G'')$ where G'' is the graph before normalisation:

For $x := \text{null}$ and $x := y$, they behave just like standard assignments; only the env mapping changes in both domains. The same bijection m works, and normalisation soundly handles any nodes that consequently lose their variable names, so these commands are sound.

For $x := \text{new}$, the concrete semantics allocate a fresh, unshared one-cell segment pointing to null. Extending m with $w \mapsto c_{\text{fresh}}$ perfectly mirrors the abstract addition of node w with $\text{succ}(w) = \text{null}$, $\text{len}(w) = 1$, and $gp(w) = ff$, and thus is sound.

As for $x := y.n$, let $m(u) \rightarrow^k m(v)$ with $k \in \llbracket l \rrbracket$. If $k = 1$, the function is identical to a standard assignment, matching the abstract logic setting x to v , and thus sound. However, when $k \geq 2$, the concrete successor is an unnamed cell c where $m(u) \rightarrow^1 c \rightarrow^{k-1} m(v)$. Since $k - 1 \in \llbracket l \ominus 1 \rrbracket$, materialising c into a fresh abstract node w with length $l \ominus 1$ exactly bounds the possible concrete states, preserving well-formedness. Thus, $x := y.n$ is sound.

The transform $x.n := y$ behaves similarly. Let $c_u = m(u)$ and $c_v \rightarrow^k m(v)$. Overwriting $n(c_u)$ conceptually orphans the $k - 1$ intermediate cells. If $k \geq 2$, the last orphaned cell still points to $m(v)$, which the abstract perfectly captures by setting $gp(v) := tt$. Attaching to $m(t)$ violates the well-formed unshared property if and only if some other cell points to $m(t)$. In the abstract, this matches exactly to t having a predecessor or $gp(t) = tt$. It violates acyclicity if and only if $m(t) \rightarrow^* c_u$, matching $t \rightarrow^* u$ in G . Otherwise, the valid mutation is accurately described by setting $\text{succ}(u) = t$, $\text{len}(u) = 1$, and thus $x.n := y$ is sound.

Since all of the abstract error checks directly correspond to concrete violations, and valid mutations preserve the bijection under normalisation, we can expand the property to the entire domain, meaning we proved that all the transfer functions are sound, and thus the analysis is sound.

Lastly, we need to prove termination. As we already demonstrated, the functions are monotone, so we need to prove that they satisfy the Ascending Chain Condition (ACC). For the shape graph, the lattice that we have chosen has a finite height. A state can ascend a finite amount of times before reaching $\top$, because a graph has at most $|Var|$ nodes, and the components $(\text{env}, \text{succ}, \text{len}, gp)$ have a strictly bounded number of combinations. That puts a limit on the length of an ascending chain, making it finite. Since the functions are monotone, any ascending chain created using the functions is finite, confirming the ACC and resulting in termination for all functions.